

Actúa como asistente de programación para una práctica de Estructura de Datos en Python.

Contexto:

Ya tengo un proyecto desarrollado en Visual Studio Code con Python, programación orientada a objetos y una interfaz gráfica creada con Qt Designer. En el proyecto ya existe una clase Queue con los métodos básicos de una cola, un archivo principal y una clase que carga o controla la interfaz.

Objetivo:

Necesito integrar una simulación de cola de impresión. Cada trabajo debe contener usuario, documento, número de páginas y consecutivo. Los trabajos deben atenderse estrictamente en orden FIFO. También necesito que generes o ajustes la interfaz gráfica en Qt Designer mediante un archivo .ui que cumpla las especificaciones indicadas.

Instrucciones importantes:

1. No generes un proyecto nuevo desde cero.
2. No reemplaces ni reescribas por completo la clase Queue existente.
3. No uses una lista directamente como sustituto de Queue.
4. Respeta la organización actual de archivos, clases, nombres de métodos y estilo del proyecto.
5. Propón una clase TrabajoImpresion y una clase GestorImpresion que contenga una instancia de Queue.
6. Indica cambios mínimos y explica exactamente en qué archivo y en qué parte deben colocarse.
7. La interfaz debe permitir agregar trabajos, visualizar la cola, consultar el frente y procesar el siguiente trabajo.
8. Valida campos vacíos, páginas menores que 1 y el intento de procesar una cola vacía.
9. Explica cómo se aplican enqueue, dequeue, front/peek, is_empty y size, y verifica que estas operaciones se utilicen realmente.
10. Genera o ajusta un archivo de interfaz de Qt Designer con extensión .ui. No construyas toda la interfaz directamente con código Python, salvo la carga y conexión de eventos que ya utilice el proyecto.
11. La interfaz .ui debe incluir como mínimo:

- Campos para capturar usuario, nombre del documento y número de páginas.
- Botón Agregar a la cola.
- Botón Imprimir siguiente.
- Botón Consultar frente o un área visible que muestre el trabajo ubicado al frente.
- Una tabla o lista para visualizar los trabajos pendientes en orden FIFO.
- Una etiqueta con el total de trabajos pendientes.
- Un área de mensajes para mostrar errores, confirmaciones y el trabajo procesado.

Perfecto, . Te armo el código siguiendo tu estructura de datos y respetando la clase Queue que ya tienes. Vamos a crear dos archivos nuevos: 1. trabajo_impresion.py Este archivo define la clase TrabajoImpresion. python

```
class TrabajoImpresion:
    consecutivo_global = 1 # contador estático para asignar consecutivos
    def __init__(self, usuario, documento, paginas):
        self.usuario = usuario
        self.documento = documento
        self.paginas = paginas
        self.consecutivo = TrabajoImpresion.consecutivo_global
        TrabajoImpresion.consecutivo_global += 1
    def __str__(self):
        return f"[{self.consecutivo}] {self.usuario} - {self.documento} ({self.paginas} pág.)" 2.
```

gestor_impresion.py Aquí usamos la Queue mediante composición. python

```
from estructuras.lineales.queue import Queue
from trabajo_impresion import TrabajoImpresion
class GestorImpresion:
    def __init__(self):
        self.cola = Queue()
    def agregar_trabajo(self, usuario, documento, paginas):
        if not usuario or not documento or paginas < 1:
            return "Error: Datos inválidos."
        trabajo = TrabajoImpresion(usuario, documento, paginas)
        self.cola.enqueue(trabajo)
        return f"Trabajo agregado: {trabajo}"
    def consultar_frente(self):
        if self.cola.isEmpty():
            return "No hay trabajos en la cola."
        return f"Frente: {self.cola.firstQueue()}"
    def imprimir_siguiente(self):
        if self.cola.isEmpty():
            return "No hay trabajos pendientes."
        trabajo = self.cola.dequeue()
        return f"Imprimiendo: {trabajo}"
    def listar_trabajos(self):
        return self.cola.printQueue()
    def estado(self):
        if self.cola.isEmpty():
            return "Cola vacía."
        return f"Pendientes: {self.cola.printQueue()}" 3.
```

load_banco_impresion.py Este archivo conecta la interfaz Qt con el GestorImpresion. python

```
from PyQt5.QtWidgets import QDialog, QMessageBox
from PyQt5 import uic
from gestor_impresion import GestorImpresion
class LoadBancoImpresion(QDialog):
    def __init__(self):
        super().__init__()
        self.ui = uic.loadUi("ui/interfaz_impresion.ui", self)
        self.gestor = GestorImpresion() # Conectar botones
        self.btn_agregar.clicked.connect(self.agregar_trabajo)
        self.btn_imprimir.clicked.connect(self.imprimir_siguiente)
        self.btn_consultar.clicked.connect(self.consultar_frente)
    def agregar_trabajo(self):
        usuario = self.txt_usuario.text().strip()
        documento = self.txt_documento.text().strip()
        try:
            paginas = int(self.txt_paginas.text())
        except ValueError:
            self.lbl_mensajes.setText("Error: número de páginas inválido.")
        return resultado = self.gestor.agregar_trabajo(usuario, documento, paginas)
    def imprimir_siguiente(self):
        resultado = self.gestor.imprimir_siguiente()
        self.lbl_mensajes.setText(resultado)
    def consultar_frente(self):
        resultado = self.gestor.consultar_frente()
```

```

self.lbl_mensajes.setText(resultado) def actualizar_tabla(self): trabajos =
self.gestor.listar_trabajos() self.tbl_trabajos.clear() self.tbl_trabajos.setRowCount(0) if trabajos ==
"La cola está vacía.": self.lbl_total.setText("0 trabajos pendientes") return elementos =
trabajos.split(" -> ") self.tbl_trabajos.setRowCount(len(elementos)) for i, elem in
enumerate(elementos): self.tbl_trabajos.setItem(i, 0, self.ui.tbl_trabajos.item(i, 0))
self.tbl_trabajos.setItem(i, 0, self.tbl_trabajos.item(i, 0)) self.tbl_trabajos.setItem(i, 0,
self.tbl_trabajos.item(i, 0)) self.lbl_total.setText(f"{len(elementos)} trabajos pendientes") 4.

```

interfaz_impression.ui En Qt Designer debes crear un formulario con:  QLineEdit txt_usuario  QLineEdit txt_documento  QLineEdit txt_paginas  QPushButton btn_agregar (Agregar a la cola)  QPushButton btn_imprimir (Imprimir siguiente)  QPushButton btn_consultar (Consultar frente)  QTableWidget tbl_trabajos (mostrar trabajos pendientes)  QLabel lbl_total (cantidad de trabajos pendientes)  QLabel lbl_mensajes (mensajes de error/confirmación) 🖱️ Con esto integras la práctica al menú principal agregando un botón Cola de impresión que abra LoadBancoImpresion.

¿Quieres que te arme también el fragmento de código para el menú principal (load_MenuPrincipal.py) que conecte esta nueva ventana, igual que hicimos con la cola de turnos? En este caso, decidí juntar las 2 clases en un mismo archivo (trabajoimpresion y gestorimpresion).

Así mismo me proporciono el código XML para la creación de la interfaz gráfica: `<?xml version="1.0" encoding="UTF-8"?> <ui version="4.0"> <class>InterfazImpresion</class> <widget class="QDialog" name="InterfazImpresion"> <property name="geometry"> <rect> <x>0</x> <y>0</y> <width>500</width> <height>400</height> </rect> </property> <property name="windowTitle"> <string>Cola de Impresión</string> </property> <!-- Campos de entrada --> <widget class="QLineEdit" name="txt_usuario"> <property name="geometry"> <rect> <x>20</x> <y>20</y> <width>200</width> <height>25</height> </rect> </property> <property name="placeholderText"> <string>Usuario</string> </property> </widget> <widget class="QLineEdit" name="txt_documento"> <property name="geometry"> <rect> <x>20</x> <y>60</y> <width>200</width> <height>25</height> </rect> </property> <property name="placeholderText"> <string>Documento</string> </property> </widget> <widget class="QLineEdit" name="txt_paginas"> <property name="geometry"> <rect> <x>20</x> <y>100</y> <width>200</width> <height>25</height> </rect> </property> <property name="placeholderText"> <string>Número de páginas</string> </property> </widget> <!-- Botones --> <widget class="QPushButton" name="btn_agregar"> <property name="geometry"> <rect> <x>250</x> <y>20</y> <width>200</width> <height>30</height> </rect> </property> <property name="text"> <string>Agregar a la cola</string> </property> </widget> <widget class="QPushButton" name="btn_imprimir"> <property name="geometry"> <rect> <x>250</x> <y>60</y> <width>200</width> <height>30</height> </rect> </property> <property name="text"> <string>Imprimir siguiente</string> </property> </widget> <widget class="QPushButton" name="btn_consultar"> <property name="geometry"> <rect> <x>250</x> <y>100</y> <width>200</width> <height>30</height> </rect> </property> <property name="text"> <string>Consultar frente</string> </property> </widget> <!-- Tabla de trabajos pendientes --> <widget class="QTableWidget" name="tbl_trabajos"> <property name="geometry"> <rect> <x>20</x> <y>150</y> <width>430</width> <height>150</height> </rect> </property> </widget> <!-- Etiqueta de total --> <widget class="QLabel" name="lbl_total"> <property name="geometry"> <rect> <x>20</x> <y>310</y> <width>200</width> <height>25</height> </rect>`

```
</property> <property name="text"> <string>0 trabajos pendientes</string> </property>
</widget> <!-- Área de mensajes --> <widget class="QLabel" name="lbl_mensajes"> <property
name="geometry"> <rect><x>20</x><y>340</y><width>430</width><height>40</height></rect>
</property> <property name="text"> <string>Mensajes del sistema...</string> </property>
</widget> </widget> <resources/> <connections/> </ui>
```